

Challenge: Building a house data UI

[< Previous](#)[Overview: Dynamic scripting with JavaScript](#)[Next >](#)

In this challenge we are going to get you to write some JavaScript for a house search/filter page on a property website. This will include fetching JSON data, filtering that data based on the values entered in provided form controls, and rendering that data to the UI. Along the way, we'll also test your knowledge of conditionals, loops, arrays and array methods, and more besides.

Starting point

To begin, click the **Play** button in one of the code panels below to open the provided example in the MDN Playground. You'll then follow the instructions in the [Project brief](#) section to complete the JavaScript functionality.

HTML

```
<h1>House search</h1>
<p>
  Search for houses for sale. You can filter your search by street, number of
  bedrooms, and number of bathrooms, or just submit the search with no filters
  to display all available properties.
</p>
<form>
  <div>
    <label for="choose-street">Street:</label>
    <select id="choose-street" name="choose-street">
      <option value="">No street selected</option>
    </select>
  </div>
  <div>
    <label for="choose-bedrooms">Number of bedrooms:</label>
    <select id="choose-bedrooms" name="choose-bedrooms">
      <option value="">Any number of bedrooms</option>
    </select>
  </div>
  <div>
    <label for="choose-bathrooms">Number of bathrooms:</label>
    <select id="choose-bathrooms" name="choose-bathrooms">
      <option value="">Any number of bathrooms</option>
```

```
    </select>
  </div>
  <div>
    <button>Search for houses</button>
  </div>
</form>
<p id="result-count">Results returned: 0</p>
<section id="output"></section>
```

JS

```
const streetSelect = document.getElementById("choose-street");
const bedroomSelect = document.getElementById("choose-bedrooms");
const bathroomSelect = document.getElementById("choose-bathrooms");
const form = document.querySelector("form");

const resultCount = document.getElementById("result-count");
const output = document.getElementById("output");

let houses;

function initializeForm() {

}

function renderHouses(e) {
  // Stop the form submitting
  e.preventDefault();

  // Add rest of code here
}

// Add a submit listener to the <form> element
form.addEventListener("submit", renderHouses);

// Call fetchHouseData() to initialize the app
fetchHouseData();
```

House search

Search for houses for sale. You can filter your search by street, number of bedrooms, and number of bathrooms, or just submit the search with no filters to display all available properties.

Street:

Number of bedrooms:

Number of bathrooms:

Results returned: 0

Project brief

You have been provided with an HTML index page containing a form allowing the user to search for houses by street, number of bedrooms, and number of bathrooms, plus a couple of elements to contain search results. You've also been provided with a JavaScript file containing some constant and variable definitions, plus a couple of skeleton function definitions. Your job is to fill in the missing JavaScript to make the house search interface work.

The provided constant and variable definitions hold the following references:

- `streetSelect`: The "choose-street" `<select>` element.
- `bedroomSelect`: The "choose-bedrooms" `<select>` element.
- `bathroomSelect`: The "choose-bathrooms" `<select>` element.
- `form`: The overall `<form>` element that contains the `<select>` elements.
- `resultCount`: The "result-count" `<p>` element, which updates to display the number of results returned after each search.
- `output`: The "output" `<section>` element, which displays the search results.
- `houses`: Initially empty, but this will contain the house data object created by parsing the fetched JSON data.

The skeleton functions are:

- `initializeForm()`: This will query the data and populate the `<select>` elements with the possible values that could be searched for.
- `renderHouses()`: This will filter the data based on the `<select>` element values, and render the results.

Fetching the data

The first thing you need to do is create a new function to fetch the house data and store it in the `houses` variable.

To do so:

1. Create a new function just below the variable and constant definitions called `fetchHouseData()`.
2. Inside the body of the function, use the `fetch()` method to fetch the JSON found at <https://mdn.github.io/shared-assets/misc/houses.json>. You should study the structure of this data in preparation for some of the later steps.
3. When the resulting promise resolves, check the `ok` property of the response. If it is `false`, throw a custom error that reports the response's `status`.
4. Provided the response is ok, return the response as JSON using the `json()` method.
5. When the resulting promise resolves, set the `houses` variable equal to the result of the `json()` method (this should be an array of objects containing house data), and call the `initializeForm()` function.

Completing the `initializeForm()` function

Now you need to write the contents of the `initializeForm()` function. This will query the data stored inside `houses` and use it to populate the `<select>` elements with `<option>` elements representing all of the different values that could be filtered for. At the moment, the `<select>` elements only contain a single `<option>` element with a value of `""` (an empty string), which represents all values. The user can choose this option if they don't want the results to be filtered by that field.

Inside the function body, write code that does the following:

1. Create `<option>` elements for all the different street names inside the "choose-street" `<select>`. There are a few ways you could do this, but we'd recommend creating a temporary array then looping through all the objects inside `houses`. Inside the loop, check whether your temporary array includes the `street` property of the current house. If not, add it to the temporary array and add an `<option>` to the "choose-street" `<select>` that includes the `street` property as its value.
2. Create options for all the possible bedroom number values inside the "choose-bedrooms" `<select>`. To do this, you could loop through the `houses` array and determine what the largest `bedrooms` value is, then write a second loop that adds an `<option>` to the "choose-bedrooms" `<select>` for each number from `1` to that largest value.
3. Create options for all the possible bathroom number values inside the "choose-bathrooms" `<select>`. This can be solved using the same technique as the previous step.

Note: You *could* just hardcode the `<option>` elements inside the HTML, but that would only work for this exact data set. We want you to write JavaScript that will correctly populate the form regardless of the data values provided (each house object would have to have the same structure).

Note: You could use the `innerHTML` property to add child content inside HTML elements, but we'd recommend not doing so. You can't always trust the data you are adding to your page: If it is not properly sanitized on the server, bad actors could use `innerHTML` as a pathway to carry out [Cross-site scripting \(XSS\)](#) attacks on your page. A safer route is to use DOM scripting features such as `createElement()`, `appendChild()`, and `textContent`. Using `innerHTML` to remove child content isn't such an issue.

Completing the `renderHouses()` function

Next, you need to complete the `renderHouses()` function body. This will filter the data based on the `<select>` element values, and render the results to the UI.

1. First, you need to filter the data. This is probably best achieved using the array `filter()` method, which returns a new array containing only the array elements that match the filter criteria.
 1. This is a fairly complex `filter()` function to write. You need to test whether the `street` property of the house is equal to the selected value of the "choose-street" `<select>`, and whether the `bedrooms` property of the house is equal to the selected value of the "choose-bedrooms" `<select>`, and whether the `bathrooms` property of the house is equal to the selected value of the "choose-bathrooms" `<select>`.
 2. Each component of the test always needs to return `true` if the associated `<select>` value is `""` (the empty string, which represents all values). You can achieve this by "short-circuiting" each check.
 3. You also need to make sure the data types match in each check. The value of a form element is always a string. This is not necessarily the case for your object property values. How can you make the data types match up for the purposes of the test?
2. Output the number of filtered search results into the "result-count" `<p>` element, using the string structure "Results returned: number".
3. Empty the "output" `<section>` element, so it doesn't have any child HTML elements. If you don't do this, each time a search is performed the results will be added to the end of the previous results rather than replacing them.
4. Create a new function inside `renderHouses()` called `renderHouse()`. This function needs to take a house object as an argument, and do two things:
 1. Calculate the total area of the rooms contained inside the house's `room_sizes` object. This isn't as straightforward as looping through an array of numbers and summing them, but it isn't too tricky.
 2. Add an `<article>` element inside the "output" `<section>` element containing the house's number, street name, bedroom and bathroom count, total room area, and price. you can vary the structure if you like, we we'd like it to be similar to this HTML snippet:

HTML

```
<article>
  <h2>number street name</h2>
  <ul>
    <li>🛏 Bedrooms: number</li>
    <li>🚿 Bathrooms: number</li>
    <li>Room area: number m2</li>
    <li>Price: £price</li>
  </ul>
</article>
```

5. Loop through all of the houses inside the filtered array and pass each one into a `renderHouse()` call.

Hints and tips

- You don't need to alter the HTML or CSS in any way.
- For doing things like finding the largest value in an array of values, the `reduce()` array function is really handy. We've not taught it in this course, as it is quite complex, but it is really powerful when you get to grips with it. As a stretch goal, try researching it and using it in your answer.

Example

Your finished app should work like the following live example:

House search

Search for houses for sale. You can filter your search by street, number of bedrooms, and number of bathrooms, or just submit the search with no filters to display all available properties.

Street:

Number of bedrooms:

Number of bathrooms:

Results returned: 2

33 Tavistock Road

 Bedrooms: 2
 Bathrooms: 2
Room area: 94m²
Price: £300000

85 Tavistock Road

 Bedrooms: 2
 Bathrooms: 1
Room area: 75m²
Price: £290000

▼ [Click here to show the solution](#)

The finished JavaScript should look something like this:

JS

```

const streetSelect = document.getElementById("choose-street");
const bedroomSelect = document.getElementById("choose-bedrooms");
const bathroomSelect = document.getElementById("choose-bathrooms");
const form = document.querySelector("form");
const resultCount = document.getElementById("result-count");
const output = document.getElementById("output");

let houses;

// Solution: Fetching the data

function fetchHouseData() {
  fetch("https://mdn.github.io/shared-assets/misc/houses.json")
    .then((response) => {
      if (!response.ok) {
        throw new Error(`HTTP error: ${response.status}`);
      }

      return response.json();
    })
    .then((json) => {
      houses = json;
      initializeForm();
    });
}

// Solution: Completing the initializeForm() function

function initializeForm() {
  // Create options for all the different street names
  const streetArray = [];
  for (let house of houses) {
    if (!streetArray.includes(house.street)) {
      streetArray.push(house.street);
      streetSelect.appendChild(document.createElement("option")).textContent =
        house.street;
    }
  }

  // Create options for all the possible bedroom values
  const largestBedrooms = houses.reduce(
    (largest, house) => (house.bedrooms > largest ? house.bedrooms : largest),
    houses[0].bedrooms,
  );
  let i = 1;

```

```

while (i <= largestBedrooms) {
  bedroomSelect.appendChild(document.createElement("option")).textContent = i;
  i++;
}

// Create options for all the possible bathroom values
const largestBathrooms = houses.reduce(
  (largest, house) => (house.bathrooms > largest ? house.bathrooms : largest),
  houses[0].bathrooms,
);
let j = 1;
while (j <= largestBathrooms) {
  bathroomSelect.appendChild(document.createElement("option")).textContent =
    j;
  j++;
}
}

// Solution: Completing the renderHouses() function

function renderHouses(e) {
  // Stop the form submitting
  e.preventDefault();

  // Filter the data
  const filteredHouses = houses.filter((house) => {
    // prettier-ignore
    const test = (streetSelect.value === "" ||
      house.street === streetSelect.value) &&
      (bedroomSelect.value === "" ||
        String(house.bedrooms) === bedroomSelect.value) &&
      (bathroomSelect.value === "" ||
        String(house.bathrooms) === bathroomSelect.value);

    return test;
  });

  // Output the result count to the "result-count" paragraph
  resultCount.textContent = `Results returned: ${filteredHouses.length}`;

  // Empty the output element
  output.innerHTML = "";

  // Create renderHouse() function
  function renderHouse(house) {
    // Calculate total room size
    let totalArea = 0;

```

```

const keys = Object.keys(house.room_sizes);
for (let key of keys) {
  totalArea += house.room_sizes[key];
}

// Output house to UI
const articleElem = document.createElement("article");
articleElem.appendChild(document.createElement("h2")).textContent =
  `${house.house_number} ${house.street}`;
const listElem = document.createElement("ul");
listElem.appendChild(document.createElement("li")).textContent =
  `🛏 Bedrooms: ${house.bedrooms}`;
listElem.appendChild(document.createElement("li")).textContent =
  `🛀 Bathrooms: ${house.bathrooms}`;
listElem.appendChild(document.createElement("li")).textContent =
  `Room area: ${totalArea}m2`;
listElem.appendChild(document.createElement("li")).textContent =
  `Price: £${house.price}`;
articleElem.appendChild(listElem);
output.appendChild(articleElem);
}

// Pass each house in the filtered array into renderHouse()
for (let house of filteredHouses) {
  renderHouse(house);
}
}

// Add a submit listener to the <form> element
form.addEventListener("submit", renderHouses);

// Call fetchHouseData() to initialize the app
fetchHouseData();

```